

Data Structures in Python

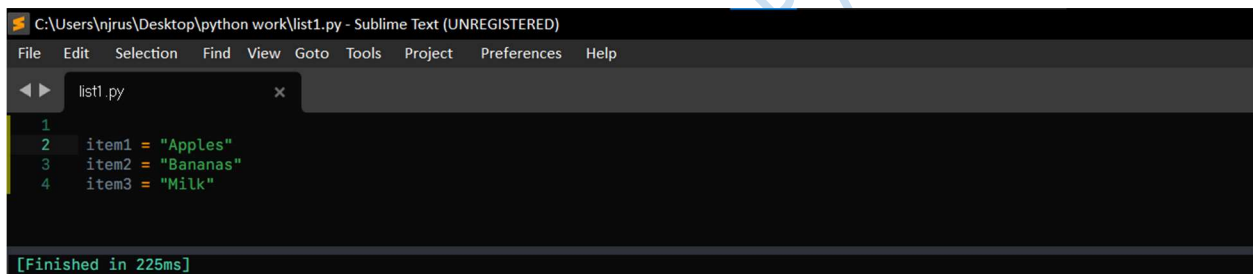
A **data structure** is a way of organizing and storing data so that a program can work with it effectively. In programming, we often need to work with more than one piece of data. Data structures give us different ways to **store, organize, and manage collections of data** depending on what we need to do.

The four main Python built-in data structures you shall encounter a lot are **lists, tuples, dictionaries, and sets**. We shall begin with lists then tackle the rest as soon as the preceding one is completed.

Lists in Python

A **list** is a data structure used to store a collection of items. That is, a list allows you to group related values together in a specific, orderly sequence, instead of keeping track of multiple individual items.

Imagine building an app for a grocery store or managing high scores in a video game. Without lists, you would have to create separate variables for every single item:



```
C:\Users\njrus\Desktop\python work\list1.py - Sublime Text (UNREGISTERED)
File Edit Selection Find View Goto Tools Project Preferences Help
list1.py
1
2 item1 = "Apples"
3 item2 = "Bananas"
4 item3 = "Milk"
[Finished in 225ms]
```

As your data grows to dozens, hundreds, or thousands of items, managing individual variable names becomes messy, repetitive, and impossible to manage dynamically. Lists solve this problem by providing a single, scalable container that can hold as many or as few items as needed.

Creating a List in Python

Creating a list in Python is simple. You write square brackets `[]` and separate each item that you manually insert with a comma.

The items inside the square brackets are called **elements (or items)** of the list. A list can contain zero items, one item, or many items.



Empty List

```
C:\Users\njrus\Desktop\python work\list1.py - Sublime Text (UNREGISTERED)
File Edit Selection Find View Goto Tools Project Preferences Help
list1.py x
1
2 # -- Empty list
3
4 shopping_items = []
5
6 print(shopping_items)

[]
[Finished in 367ms]
```

List with Items

```
C:\Users\njrus\Desktop\python work\list1.py - Sublime Text (UNREGISTERED)
File Edit Selection Find View Goto Tools Project Preferences Help
list1.py x
1
2 # -- List with items
3
4 fruits = ['orange', 'apple', 'pineapple']
5
6 print(fruits)

['orange', 'apple', 'pineapple']
[Finished in 239ms]
```

```
C:\Users\njrus\Desktop\python work\list1.py - Sublime Text (UNREGISTERED)
File Edit Selection Find View Goto Tools Project Preferences Help
list1.py x
1
2 # -- List with items
3
4 temperatures = [45, 24, 56, 76]
5
6 print(temperatures)

[45, 24, 56, 76]
[Finished in 191ms]
```

Some Key Characteristics of Python Lists

Lists Can Contain Duplicate Values

Unlike some other data structures, lists do not require uniqueness. The same value can appear multiple times. This is useful when repetition is meaningful. For example, a list of test scores may contain the same score more than once.

See below:



```
C:\Users\njrus\Desktop\python work\list1.py - Sublime Text (UNREGISTERED)
File Edit Selection Find View Goto Tools Project Preferences Help
list1.py x
1
2 # -- Lists can have duplicate items
3
4 scores = [100, 85, 90, 100, 85]
5
6 print(scores)

[100, 85, 90, 100, 85]
[Finished in 216ms]
```

Lists Can Contain Different Types of Values

Python's lists are heterogeneous. This means that you can mix different data types within the same list:

```
C:\Users\njrus\Desktop\python work\list1.py - Sublime Text (UNREGISTERED)
File Edit Selection Find View Goto Tools Project Preferences Help
list1.py x
1
2 # -- Lists can contain different data types
3
4 user_profile = ["Alice", 28, True, 98.5]
5
6 print(user_profile)

['Alice', 28, True, 98.5]
[Finished in 193ms]
```

For example, a single list could contain a name, a number, and a Boolean value. Python allows this because lists are not restricted to one data type.

However, while a list can store unrelated items, it is almost always used to group values that share a **common context** or **purpose**. For example:

- A list of user names in an application
- A collection of daily temperature readings
- A list of active shopping cart items

This is important as you would want to make your programs cleaner, easier to understand, and much more manageable.

Lists Are Ordered and Changeable

Ordered: The items in a list have a fixed position. The order in which you define or add elements is preserved unless explicitly rearranged.

Changeable (Mutable): Once a list is created, you can modify it by adding new items, removing existing ones, or changing values without creating another list.



FROM BASICS TO JUNIOR PYTHON DEV: Data Structures: Lists

Use Cases of Lists from Everyday Programming

Industry / Context	Use Case	Example List Concept
E-commerce	Shopping Cart	Selected items pending checkout
Social media	Content Feed	Recent posts, updates, or followers
Gaming	Leaderboards & Inventories	Player items, high scores, or active quests
Task Management	To-Do Applications	Pending tasks, completed milestones

We will explore how to work with lists in the upcoming lessons. We shall look at;

- **How to access an item in a list.**
- **How indexing works.**
- **How to change an item.**
- **How to add and remove items.**
- **Useful list methods.**

